Name: Omkar Gholap
Class: D20A
Roll No: 9
BlockChain & DLT

EXPERIMENT NO - 8

AIM: Implement the Blockchain platform Ganache

Theory:

Ganache is a personal Ethereum blockchain used for testing and development. It allows developers to deploy contracts, develop decentralized applications (DApps), and run tests in a safe and deterministic environment.

Ganache provides:

- Instant mining of blocks
- Pre-funded test accounts
- Transaction history view
- Graphical interface to monitor blockchain activities

It is part of the Truffle Suite and is commonly used with Remix and Metamask for smart contract deployment.

Steps to Connecat Ganache with Metamask and Remix IDE:

1. Install Ganache:
    - Download and install Ganache from the Truffle Suite website.
    - Launch it to start a local blockchain network with test Ether accounts.
2. Connect Ganache Accounts with Metamask:
    - Open Metamask and switch to a Custom RPC Network.
    - Enter the Ganache RPC server URL (e.g., HTTP://127.0.0.1:7545).
    - Import private keys of Ganache accounts into Metamask.
3. Connect Remix IDE with Metamask:
    - Open Remix IDE in the browser.
      Choose Injected Web3 as the environment in the "Deploy & Run Transactions" tab.
    - Remix will automatically connect to the Metamask account (linked with Ganache).
4. Create a Simple Solidity Smart Contract:
    - Write a Solidity program related to your mini project (e.g., Student Record, Donation Tracker, etc.).

```
pragma solidity ^0.8.0;
    contract SimpleStorage {
      uint public data;
      function set(uint x) public {
        data = x;
```

}

5. Compile and Deploy the Contract:
   Compile using Remix.
   - Deploy via the Metamask account connected to Ganache.
   - Approve the transaction in Metamask.
6. Check Transaction Details on Ganache:
   - View mined transactions, block details, and balances in the Ganache interface.
7. Interact with the Smart Contract:
   - Use Remix to call contract functions (set() or get()).
   - Observe the state changes and corresponding transactions on Ganache.

CODE:

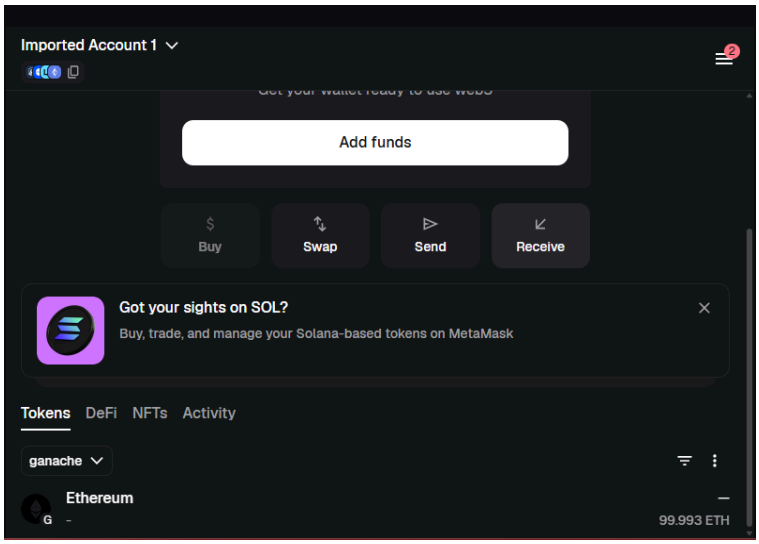
```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.19;

contract PersonalHealthRecord {

    struct Record {
        string ipfsHash;
        string recordType;
        uint256 timestamp;
        address addedBy;
    }

    struct Patient {
        string name;
        uint age;
        bool exists;
    }

    mapping(address => Patient) public patients;
    mapping(address => Record[]) private records;
    mapping(address => mapping(address => bool)) public accessPermission;

    event PatientRegistered(address patient);
    event RecordAdded(address patient, string ipfsHash);
    event AccessGranted(address patient, address doctor);
    event AccessRevoked(address patient, address doctor);

    modifier onlyPatient() {
        require(patients[msg.sender].exists, "Patient not registered");
```

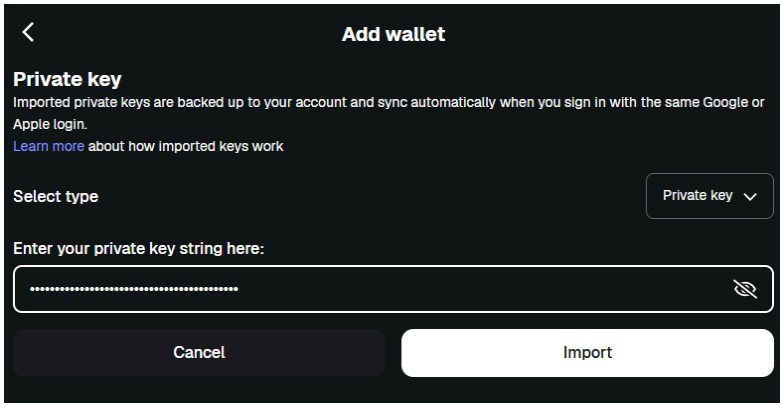
```
    _;
}

function registerPatient(string memory _name, uint _age) public {
    require(!patients[msg.sender].exists, "Already registered");

    patients[msg.sender] = Patient(_name, _age, true);

    emit PatientRegistered(msg.sender);
}

function addRecord(
    string memory _ipfsHash,
    string memory _recordType
) public onlyPatient {

    records[msg.sender].push(
        Record({
            ipfsHash: _ipfsHash,
            recordType: _recordType,
            timestamp: block.timestamp,
            addedBy: msg.sender
        })
    );

    emit RecordAdded(msg.sender, _ipfsHash);
}

function grantAccess(address doctor) public onlyPatient {
    accessPermission[msg.sender][doctor] = true;
    emit AccessGranted(msg.sender, doctor);
}

function revokeAccess(address doctor) public onlyPatient {
    accessPermission[msg.sender][doctor] = false;
    emit AccessRevoked(msg.sender, doctor);
}

function getRecords(address patient)
    public
    view
    returns (Record[] memory)
{
    require(
```

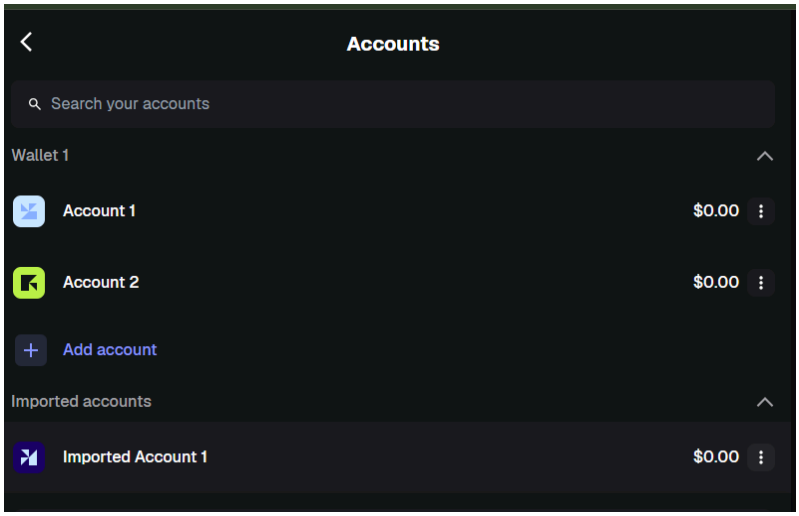
```
        patient == msg.sender || accessPermission[patient][msg.sender],
        "Access denied"
    );

    return records[patient];
  }
}
```

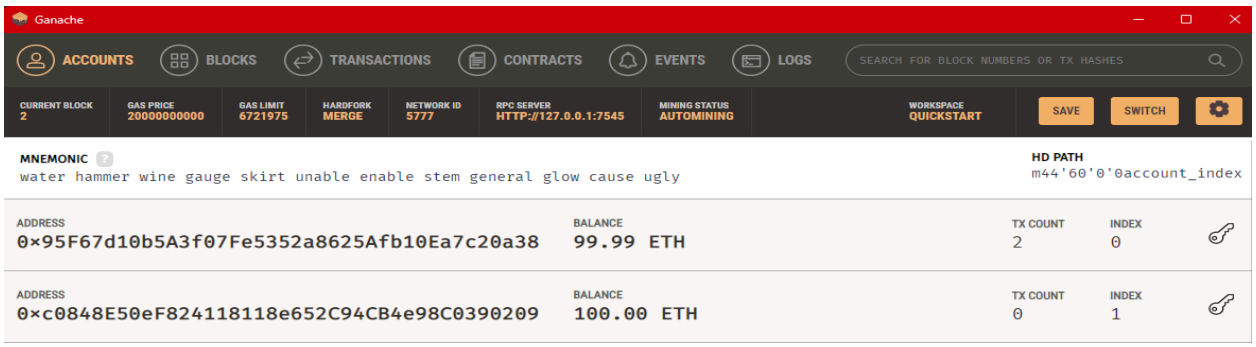
OUTPUT :

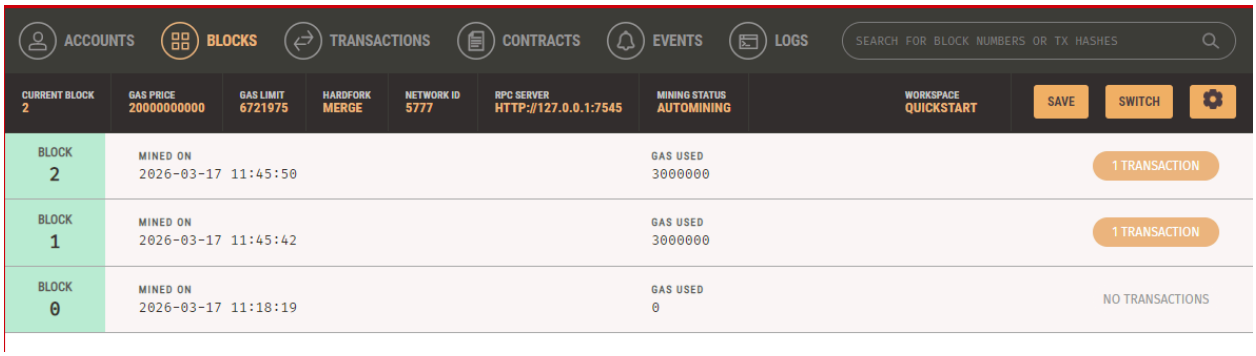
Metamask

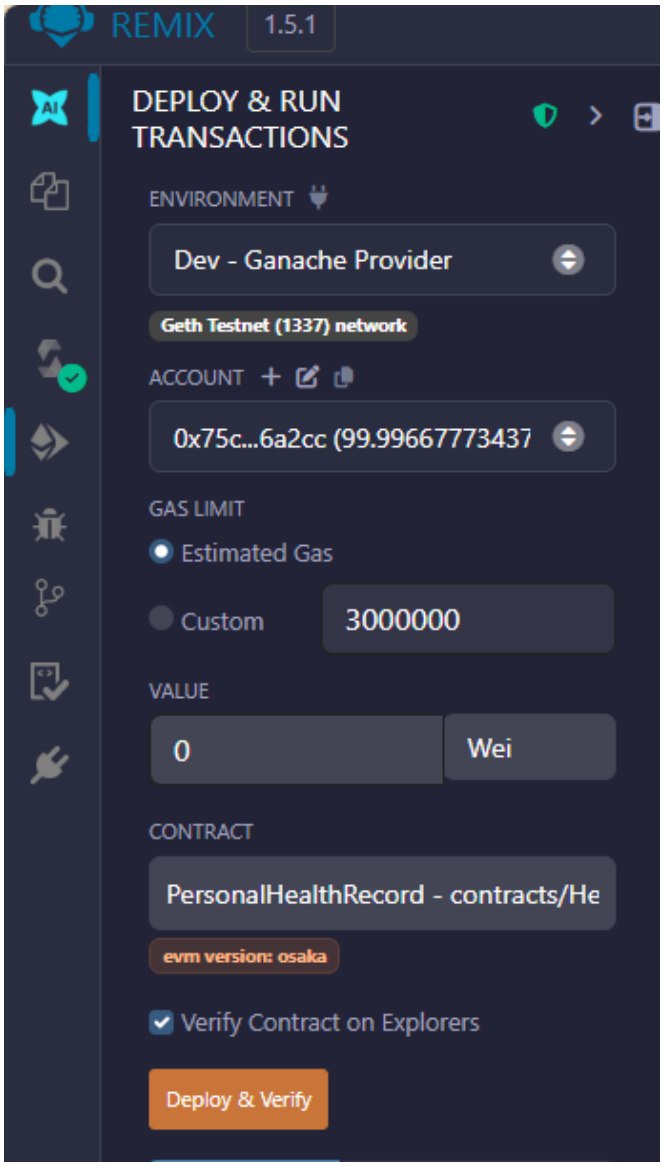





Accounts
Search your accounts
Wallet 1
Account 1
$0.00
Account 2
$0.00
Add account
Imported accounts
Imported Account 1
$0.00


Ganache
ACCOUNTS
BLOCKS
TRANSACTIONS
CONTRACTS
EVENTS
LOGS
SEARCH FOR BLOCK NUMBERS OR TX HASHES
CURRENT BLOCK 2
GAS PRICE 20000000000
GAS LIMIT 6721975
HARDFORK MERGE
NETWORK ID 5777
RPC SERVER HTTP://127.0.0.1:7545
MINING STATUS AUTOMINING
WORKSPACE QUICKSTART
SAVE
SWITCH
MNEMONIC
water hammer wine gauge skirt unable enable stem general glow cause ugly
HD PATH
m44'60'0'0account_index
ADDRESS
0x95F67d10b5A3f07Fe5352a8625Afb10Ea7c20a38
BALANCE
99.99 ETH
TX COUNT
2
INDEX
0
ADDRESS
0xc0848E50eF824118118e652C94CB4e98C0390209
BALANCE
100.00 ETH
TX COUNT
0
INDEX
1


ACCOUNTS
BLOCKS
TRANSACTIONS
CONTRACTS
EVENTS
LOGS
SEARCH FOR BLOCK NUMBERS OR TX HASHES
CURRENT BLOCK 2
GAS PRICE 20000000000
GAS LIMIT 6721975
HARDFORK MERGE
NETWORK ID 5777
RPC SERVER HTTP://127.0.0.1:7545
MINING STATUS AUTOMINING
WORKSPACE QUICKSTART
SAVE
SWITCH
BLOCK 2
MINED ON 2026-03-17 11:45:50
GAS USED 3000000
1 TRANSACTION
BLOCK 1
MINED ON 2026-03-17 11:45:42
GAS USED 3000000
1 TRANSACTION
BLOCK 0
MINED ON 2026-03-17 11:18:19
GAS USED 0
NO TRANSACTIONS


REMIX
1.5.1
DEPLOY & RUN TRANSACTIONS
ENVIRONMENT
Dev - Ganache Provider
Geth Testnet (1337) network
ACCOUNT
0x75c...6a2cc (99.99667773437
GAS LIMIT
Estimated Gas
Custom
3000000
VALUE
0
Wei
CONTRACT
PersonalHealthRecord - contracts/He
evm version: osaka
Verify Contract on Explorers
Deploy & Verify